

BÁO CÁO THỰC TẬP TUẦN 2

Chủ đề: Collections, Design Patterns & Testing

Intern: Bùi Văn Nam

Team: Platform - Adtech

Gmail: buivannam13032004@gmail.com

Leader: Nguyễn Văn Cường

1. Mục tiêu tuần

Trong tuần thứ hai, theo roadmap mục tiêu là tìm hiểu chuyên sâu về **Java Collections Framework**, các **Design Pattern** phổ biến và kiến thức cơ bản về **Testing**. Nội dung học tập hướng đến việc hiểu rõ cơ chế hoạt động của các cấu trúc dữ liệu trong Java, áp dụng các mẫu thiết kế để xây dựng phần mềm dễ mở rộng và bảo trì, đồng thời làm quen với các phương pháp kiểm thử nhằm nâng cao chất lượng mã nguồn.

Lịch học Tuần 2 (Thứ 2 - Thứ 6)

Ngày	Nội dung học	Kết quả đạt được
Thứ 2	HashMap, HashSet, ArrayList Internals	Hiểu cơ chế hoạt động bên trong của HashMap, HashSet và ArrayList; nắm được cách lưu trữ dữ liệu và hiệu năng của từng cấu trúc.
Thứ 3	HashCode, Equals & Concurrent Collections	Hiểu mối quan hệ giữa hashCode() và equals(); tìm hiểu ConcurrentHashMap và BlockingQueue trong môi trường đa luồng.
Thứ 4	Design Patterns	Nắm được mục đích và cách áp dụng các Design Pattern: Singleton, Factory, Observer và Strategy.
Thứ 5	Unit Testing & Mocking	Tìm hiểu nguyên tắc Unit Testing, sử dụng JUnit 5 và Mocking để kiểm thử các thành phần của chương trình.
Thứ 6	Integration Testing & Test Coverage	Hiểu các chiến lược Integration Testing, Test Coverage, Quality Metrics và ôn tập kiến thức tuần.

2. Chi tiết nội dung đã học

2.1 Java Collections

a. HashMap, HashSet và ArrayList Internals

Đã tìm hiểu cơ chế hoạt động bên trong của các cấu trúc dữ liệu phổ biến trong Java:

- HashMap
- HashSet
- ArrayList

Qua đó hiểu được cách dữ liệu được lưu trữ trong bộ nhớ, cơ chế mở rộng dung lượng (Resize), xử lý va chạm (Collision) của HashMap và hiệu năng của từng cấu trúc dữ liệu.

Lý thuyết:

- **ArrayList** lưu trữ dữ liệu trên mảng động, có tốc độ truy cập phần tử nhanh nhưng việc thêm hoặc xóa ở giữa danh sách có thể làm giảm hiệu năng do phải dịch chuyển các phần tử.
- **HashMap** lưu trữ dữ liệu theo cặp khóa - giá trị (Key - Value), sử dụng hàm băm (hashCode) để xác định vị trí lưu trữ nên tốc độ tìm kiếm trung bình đạt $O(1)$.
- **HashSet** được xây dựng dựa trên HashMap, chỉ lưu trữ các phần tử không trùng lặp và không đảm bảo thứ tự của dữ liệu.

b. hashCode và Equals Contract

Đã nghiên cứu nguyên tắc hoạt động của hai phương thức hashCode() và equals() trong Java.

Hiểu được mối quan hệ giữa hai phương thức, quy tắc khi Override và tầm quan trọng của chúng khi sử dụng HashMap hoặc HashSet.

Lý thuyết:

Trong Java, equals() dùng để so sánh nội dung của hai đối tượng, còn hashCode() trả về giá trị băm của đối tượng. Nếu hai đối tượng được xem là bằng nhau theo equals() thì bắt buộc phải có cùng giá trị hashCode(). Việc tuân thủ quy tắc này giúp các Collection hoạt động chính xác và đạt hiệu năng cao

c. Concurrent Collections

Đã tìm hiểu các Collection hỗ trợ lập trình đa luồng:

- ConcurrentHashMap
- BlockingQueue

Qua đó hiểu được sự khác biệt giữa Collection thông thường và Collection thread-safe.

Lý thuyết:

Concurrent Collections được thiết kế để nhiều luồng có thể truy cập và thao tác đồng thời mà vẫn đảm bảo tính nhất quán của dữ liệu. So với Hashtable hoặc Vector, các Collection này có hiệu năng tốt hơn do chỉ khóa một phần dữ liệu thay vì khóa toàn bộ cấu trúc.

2.2 Design Patterns

a. Singleton Pattern

Đã tìm hiểu Singleton Pattern và hiểu được cách đảm bảo chỉ tồn tại một đối tượng duy nhất của một lớp trong toàn bộ chương trình.

Lý thuyết:

Singleton thường được sử dụng cho các thành phần chỉ cần một đối tượng duy nhất như Logger, Configuration hoặc Database Connection Manager nhằm tiết kiệm tài nguyên và đảm bảo tính nhất quán.

b. Factory Pattern

Đã nghiên cứu Factory Pattern và hiểu được cách tạo đối tượng thông qua lớp Factory thay vì khởi tạo trực tiếp.

Lý thuyết:

Factory Pattern giúp tách quá trình khởi tạo đối tượng khỏi quá trình sử dụng, giảm sự phụ thuộc giữa các lớp và giúp chương trình dễ mở rộng khi có thêm nhiều loại đối tượng mới.

c. Observer Pattern

Đã tìm hiểu Observer Pattern và cơ chế giao tiếp giữa Subject và Observer.

Lý thuyết:

Observer Pattern xây dựng mối quan hệ một-nhiều giữa các đối tượng. Khi trạng thái của Subject thay đổi, tất cả Observer đã đăng ký sẽ được thông báo và cập nhật tự động.

d. Strategy Pattern

Đã nghiên cứu Strategy Pattern và cách thay đổi thuật toán xử lý một cách linh hoạt.

Lý thuyết:

Strategy Pattern đóng gói các thuật toán thành các lớp riêng biệt và cho phép lựa chọn thuật toán phù hợp trong quá trình chạy chương trình mà không cần sửa đổi mã nguồn hiện có.

2.3 Testing

a. Unit Testing

Đã tìm hiểu các nguyên tắc của Unit Testing và cách sử dụng **JUnit 5** để kiểm thử các phương thức trong Java.

Lý thuyết:

Unit Testing là quá trình kiểm thử từng đơn vị nhỏ nhất của chương trình (thường là một phương thức hoặc một lớp). Mục tiêu là phát hiện lỗi sớm và đảm bảo từng thành phần hoạt động đúng trước khi tích hợp với các thành phần khác.

b. Mocking

Đã nghiên cứu khái niệm Mocking và vai trò của Mock Object.

Lý thuyết:

Mocking tạo ra các đối tượng giả lập để thay thế các thành phần phụ thuộc như cơ sở dữ liệu hoặc API, giúp việc kiểm thử tập trung vào chức năng cần kiểm tra mà không bị ảnh hưởng bởi các yếu tố bên ngoài.

c. Integration Testing

Đã tìm hiểu Integration Testing và các chiến lược kiểm thử tích hợp.

Lý thuyết:

Integration Testing được thực hiện sau Unit Testing nhằm kiểm tra sự tương tác giữa nhiều module trong hệ thống, giúp phát hiện các lỗi phát sinh trong quá trình trao đổi dữ liệu hoặc gọi hàm giữa các thành phần.

d. Test Coverage và Quality Metrics

Đã nghiên cứu các chỉ số đánh giá chất lượng kiểm thử.

Lý thuyết:

- **Test Coverage** thể hiện tỷ lệ mã nguồn được thực thi bởi các bộ kiểm thử.

- **Code Coverage** đánh giá mức độ bao phủ của các câu lệnh, nhánh hoặc điều kiện trong chương trình.
- **Quality Metrics** là các chỉ số dùng để đánh giá chất lượng mã nguồn, khả năng bảo trì và hiệu quả của quá trình kiểm thử.

3. Kết quả đạt được

Sau khi hoàn thành tuần học thứ hai, đã đạt được các kết quả sau:

- Hiểu cơ chế hoạt động bên trong của HashMap, HashSet và ArrayList.
- Nắm được nguyên tắc sử dụng hashCode() và equals() trong Java.
- Hiểu sự khác biệt giữa Collection thông thường và Concurrent Collection.
- Nắm được mục đích, cách hoạt động và trường hợp áp dụng của các Design Pattern như Singleton, Factory, Observer và Strategy.
- Biết cách xây dựng Unit Test cơ bản bằng JUnit 5.
- Hiểu vai trò của Mocking trong quá trình kiểm thử phần mềm.
- Nắm được các chiến lược Integration Testing và quy trình kiểm thử tích hợp.
- Hiểu ý nghĩa của Test Coverage và các chỉ số đánh giá chất lượng mã nguồn.
- Xây dựng được nền tảng về thiết kế phần mềm và kiểm thử để chuẩn bị học các framework Backend như Spring Boot và phát triển các ứng dụng có khả năng mở rộng, dễ bảo trì.

4. Kế hoạch tuần 3

Tuần 3: Concurrency, Docker

Yêu cầu Concurrency:

- Thread lifecycle và management
- ThreadPool patterns và ExecutorService
- Synchronization: synchronized, Lock, Atomic classes
- CompletableFuture và reactive programming basics

Yêu cầu Docker:

- Container concepts và Docker architecture
- Dockerfile best practices
- Image layering và optimization
- Container networking và volumes
- Docker Compose cho multi-container apps

Lý thuyết Docker:

- **Container vs VM:** Resource usage, isolation

- **Image layers:** Union filesystem, layer caching
- **Container lifecycle:** create, start, stop, remove
- **Networking:** bridge, host, overlay networks
- **Storage:** bind mounts, volumes, tmpfs